Mounting Google Drive drive first

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
from sklearn.preprocessing import MinMaxScaler
```

Loading the dataset downloaded from kaggle

```
#Loading the dataset
train_df = pd.read_csv("/content/drive/MyDrive/dataset/DailyDelhiClimateTrain.csv", parse_dates=['date'])
test_df = pd.read_csv('/content/drive/MyDrive/dataset/DailyDelhiClimateTest.csv', parse_dates=['date'])
print("Datasets loaded successfully.")
```

```
Datasets loaded successfully.
```

1. Loading & Inspecting Data

Display information of the data:

```
# Display information of the data:
print("\nTraining Dataset Information :-")
train_df.info()
print("\nFirst 5 rows of the Training dataset:")
print(train_df.head())

print("\n--- Test Dataset Information :-")
test_df.info()
print("\nFirst 5 rows of the Test dataset:- ")
print(test_df.head())
```

```
Training Dataset Information :-
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1462 entries, 0 to 1461
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   date          1462 non-null   datetime64[ns]
 1   meantemp      1462 non-null   float64
 2   humidity      1462 non-null   float64
 3   wind_speed    1462 non-null   float64
 4   meanpressure  1462 non-null   float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 57.2 KB

First 5 rows of the Training dataset:
        date   meantemp   humidity  wind_speed  meanpressure
0 2013-01-01  10.000000  84.500000    0.000000   1015.666667
1 2013-01-02   7.400000  92.000000    2.980000   1017.800000
2 2013-01-03   7.166667  87.000000    4.633333   1018.666667
3 2013-01-04   8.666667  71.333333    1.233333   1017.166667
4 2013-01-05   6.000000  86.833333    3.700000   1016.500000

--- Test Dataset Information :-
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 114 entries, 0 to 113
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   date          114 non-null    datetime64[ns]
 1   meantemp      114 non-null    float64
 2   humidity      114 non-null    float64
 3   wind_speed    114 non-null    float64
 4   meanpressure  114 non-null    float64
dtypes: datetime64[ns](1), float64(4)
memory usage: 4.6 KB

First 5 rows of the Test dataset:-
        date   meantemp   humidity  wind_speed  meanpressure
0 2017-01-01  15.913043  85.869565    2.743478     59.000000
```

```
1 2017-01-02  18.500000  77.222222    2.894444   1018.277778
2 2017-01-03  17.111111  81.888889    4.016667   1018.333333
3 2017-01-04  18.700000  70.050000    4.545000   1015.700000
4 2017-01-05  18.388889  74.944444    3.300000   1014.333333
```

Plotting the MaxTemp column over time using matplotlib:

```
# Plotting for Training Data
plt.figure(figsize=(14, 5)) # Using a slightly different size from default
plt.plot(train_df['date'], train_df['maxtemp'], color='crimson', label='Max Temperature (Train)')
plt.title('Maximum Temperature Over Time (Training Data - Delhi)')
plt.xlabel('Date')
plt.ylabel('Max Temperature (°C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

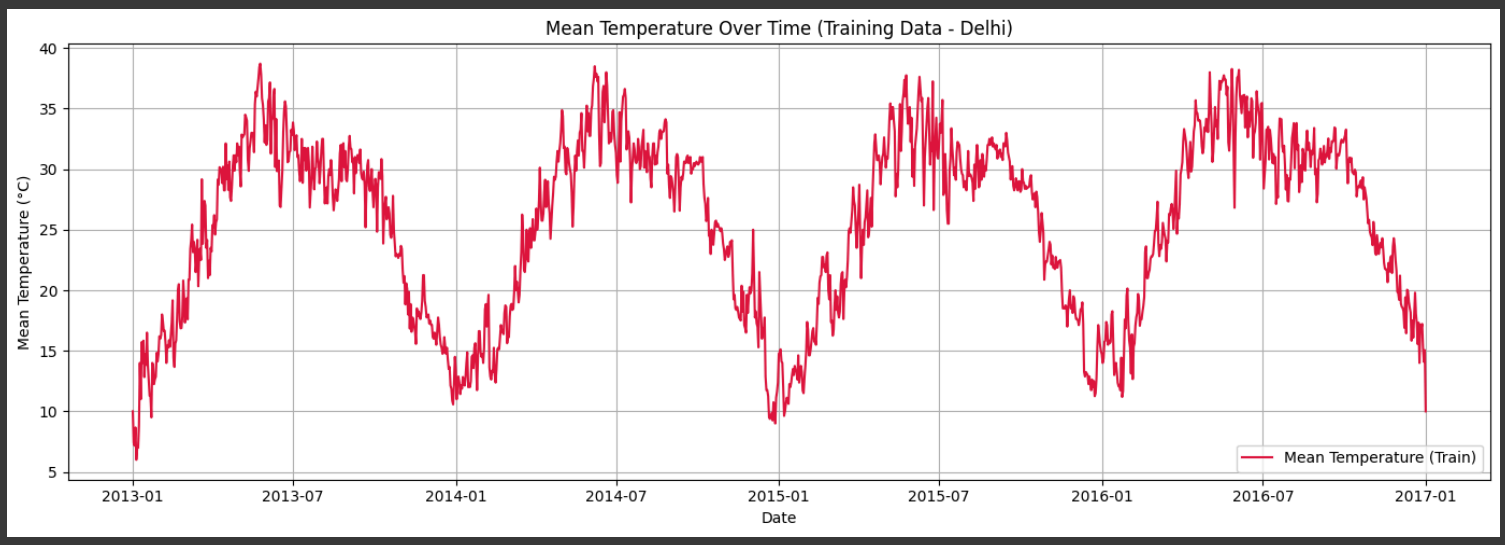
```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3804         try:
-> 3805             return self._engine.get_loc(casted_key)
   3806         except KeyError as err:

index.pyx in pandas._libs.index.IndexEngine.get_loc()

index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'maxtemp'

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
                              2 frames
/usr/local/lib/python3.11/dist-packages/pandas/core/indexes/base.py in get_loc(self, key)
   3810             ):
   3811                 raise InvalidIndexError(key)
-> 3812             raise KeyError(key) from err
   3813         except TypeError:
   3814             # If we have a listlike key, _check_indexing_error will raise

KeyError: 'maxtemp'

<Figure size 1400x500 with 0 Axes>
```

Next steps: Explain error

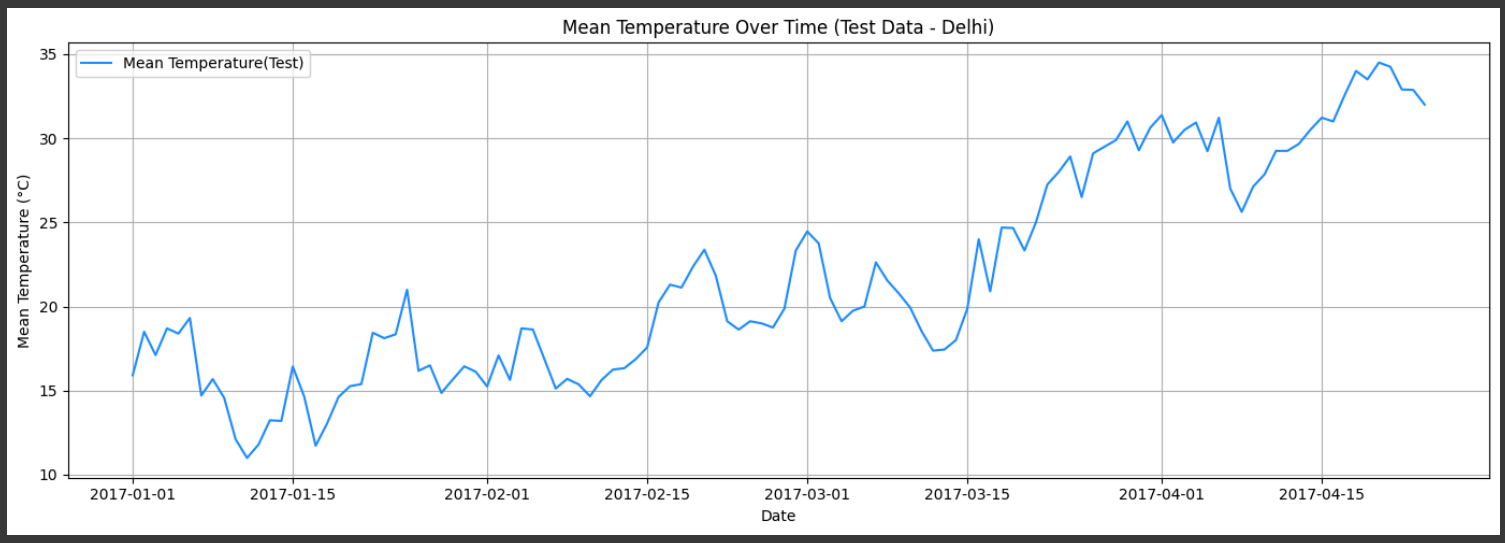
There is no column named "MaxTemp" in these CSV files.

> The only available temperature related column is "meantemp". So, I am using "meantemp", because of this, above error occured.

```
plt.figure(figsize=(14, 5)) # Using a slightly different size from default
plt.plot(train_df['date'], train_df['meantemp'], color='crimson', label='Mean Temperature (Train)')
plt.title('Mean Temperature Over Time (Training Data - Delhi)')
plt.xlabel('Date')
plt.ylabel('Mean Temperature (°C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
# Plot for Test Data
plt.figure(figsize=(14, 5))
plt.plot(test_df['date'], test_df['meantemp'], color='dodgerblue', label='Mean Temperature(Test)')
plt.title('Mean Temperature Over Time (Test Data - Delhi)')
plt.xlabel('Date')
plt.ylabel('Mean Temperature (°C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



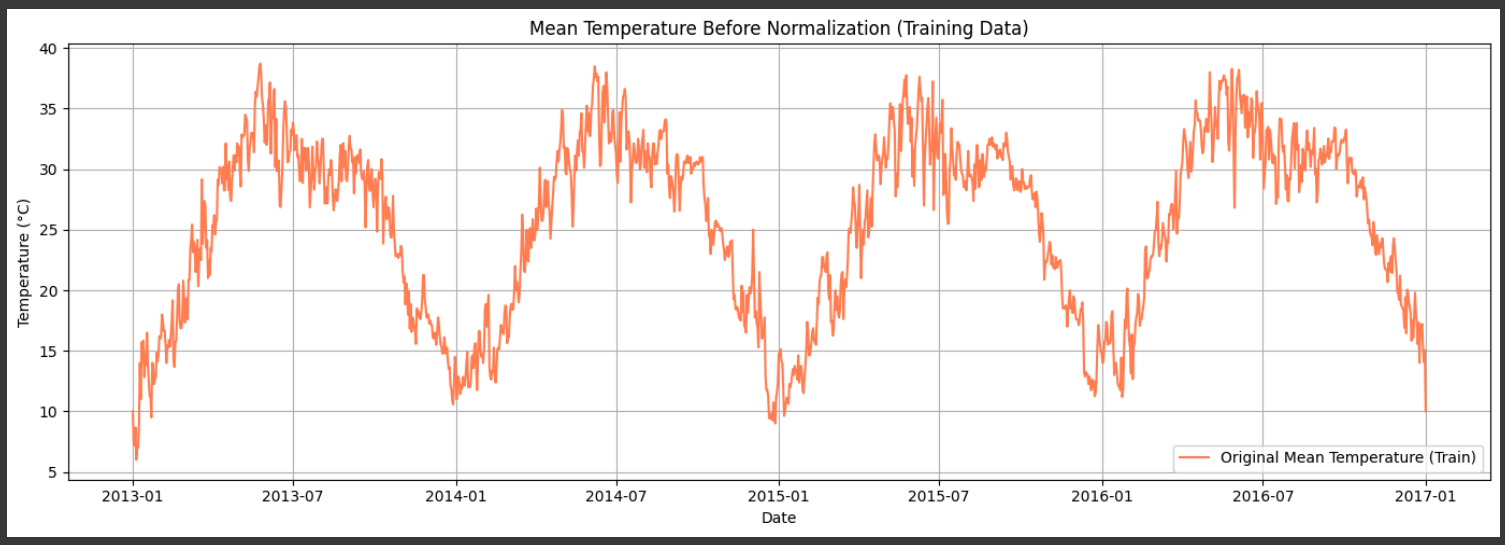
2. Preprocessing the Data

```
print("Extracting 'meantemp' column from the training data as MaxTemp data is NOT available")
```

```
Extracting 'meantemp' column from the training data as MaxTemp data is NOT available
```

```
temp_values_train = train_df['meantemp'].values.reshape(-1, 1)
```

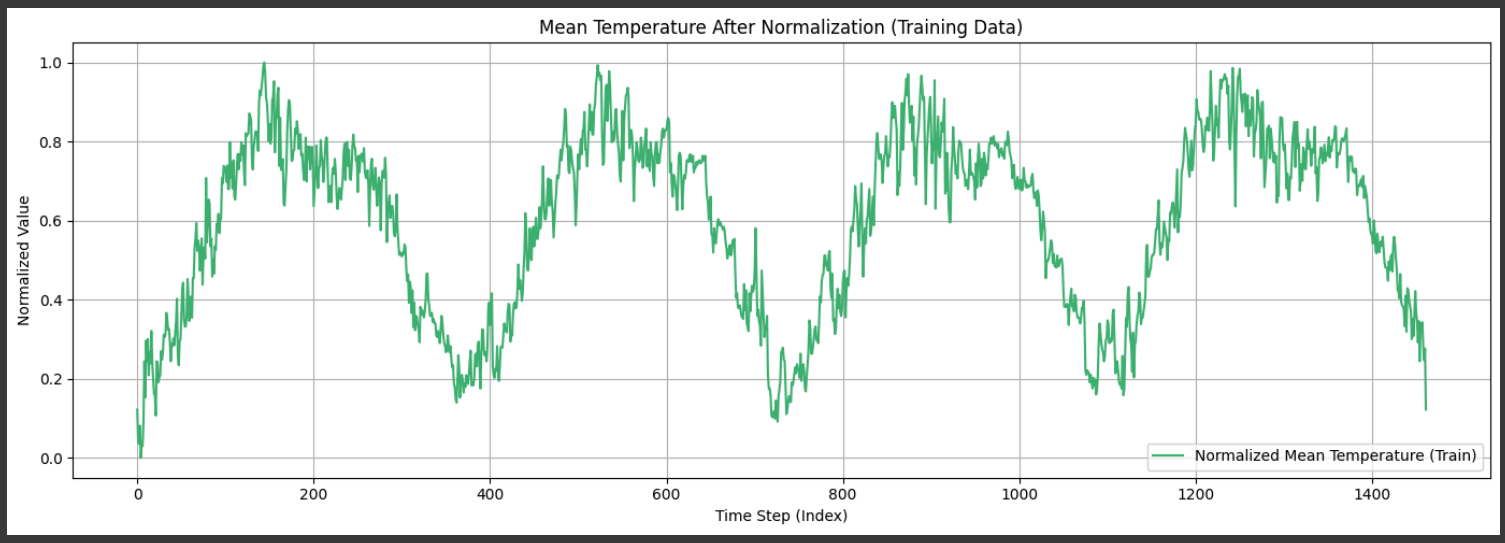
```
# Plot before normalization
plt.figure(figsize=(14, 5))
plt.plot(train_df['date'], temp_values_train, color='coral', label='Original Mean Temperature (Train)')
plt.title('Mean Temperature Before Normalization (Training Data)')
plt.xlabel('Date')
plt.ylabel('Temperature (°C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
# Normalizing it using MinMaxScaler
print("\nNormalizing the 'meantemp' data using MinMaxScaler...")
scaler = MinMaxScaler(feature_range=(0, 1)) # Scale values to be between 0 and 1
# Fitting the scaler on the training data and transform it
temp_scaled_train = scaler.fit_transform(temp_values_train)
print("Normalization complete.")
print(f"Shape of original data: {temp_values_train.shape}")
print(f"Shape of scaled data:   {temp_scaled_train.shape}")
print(f"Min value in scaled data: {temp_scaled_train.min():.2f}, Max value: {temp_scaled_train.max():.2f}")
```

```
Normalizing the 'meantemp' data using MinMaxScaler...
Normalization complete.
Shape of original data: (1462, 1)
Shape of scaled data:   (1462, 1)
Min value in scaled data: 0.00, Max value: 1.00
```

```
# Plotting after normalization
plt.figure(figsize=(14, 5))
plt.plot(temp_scaled_train, color='mediumseagreen', label='Normalized Mean Temperature (Train)')
plt.title('Mean Temperature After Normalization (Training Data)')
plt.xlabel('Time Step (Index)')
plt.ylabel('Normalized Value')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



3. Creating Sequences for Prediction

```
def create_sequences(data, seq_length=30):
    """
    Creates sequences of data for time series forecasting.
    Arguments:
    1) data (np.array): The input time series data.
    2) seq_length (int): The length of the input sequences
```

```
    2) seq_length (int): The length of the input sequences.

    Returns:
    tuple: A tuple containing:
      x (np.array): Input sequences of shape.
      y (np.array): Target values of shape.
    """
    x_data = []
    y_data = []

    for i in range(seq_length, len(data)):
        # Extracting a sequence of 'seq_length' from the data
        x_data.append(data[i-seq_length:i, 0]) # Taking the first feature
        # The value at the current time step 'i' is the target
        y_data.append(data[i, 0])             # Taking the first feature

    x_data = np.array(x_data)
    y_data = np.array(y_data)

    # 3) Reshaping x to shape (samples, seq_length, 1)
    # The '1' signifies that we have 1 feature (temperature) at each time step.
    x_data = x_data.reshape((x_data.shape[0], x_data.shape[1], 1))

    return x_data, y_data
```

```
# 2) Splitting data into x and y using the function
sequence_length = 30 # Number of past days to use for prediction
x_train_sequences, y_train_sequences = create_sequences(temp_scaled_train, seq_length=sequence_length)

print(f"\nCreated sequences with sequence_length = {sequence_length}")
print(f"Shape of x_train_sequences (input sequences): {x_train_sequences.shape}")
print(f"Shape of y_train_sequences (target values):   {y_train_sequences.shape}")
```

```
Created sequences with sequence_length = 30
Shape of x_train_sequences (input sequences): (1432, 30, 1)
Shape of y_train_sequences (target values):   (1432,)
```

```
# Printing the first sequence and its corresponding target
if len(x_train_sequences) > 0:
    print("\nExample of the first sequence (x[0]):")
    print(x_train_sequences[0].flatten()) # Flatten for easier viewing
    print(f"Corresponding target (y[0]): {y_train_sequences[0]}")
```

```
Example of the first sequence (x[0]):
[0.12227074 0.04279476 0.0356623  0.08151383 0.         0.03056769
 0.03056769 0.08733624 0.24454148 0.15283843 0.29694323 0.24454148
 0.30058224 0.20887918 0.26637555 0.23944687 0.3209607  0.23944687
 0.19868996 0.16157205 0.15895197 0.1069869  0.24454148 0.23944687
 0.19104803 0.20378457 0.20960699 0.27001456 0.24836245 0.26637555]
Corresponding target (y[0]): 0.3117903930131003
```

Building & Compiling an RNN Model

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
```

```
print("--- Q4: Build & Compile an RNN Model (Simplified) ---")

# Defining sequence_length and input_shape directly
# This assumes that the sequences created in Question 3 used a length of 30.
sequence_length = 30
input_shape_rnn = (sequence_length, 1) # (timesteps, features_per_timestep)
print(f"RNN input shape set to: {input_shape_rnn}")

# 1) Building  a model using: SimpleRNN(units=50, return_sequences=False), Dense(1) output layer

print("\nBuilding the RNN model...")
model_rnn = Sequential([
    SimpleRNN(units=50, return_sequences=False, input_shape=input_shape_rnn),
    Dense(1) # Output layer for single value prediction
])
print("Model built successfully.")

# 2) Compiling with mean_squared_error loss and adam optimizer
print("\nCompiling the model...")
model_rnn.compile(optimizer='adam', loss='mean_squared_error')
```

```
print("Model compiled successfully with 'adam' optimizer and 'mean_squared_error' loss.")

# 3) Printing model summary
print("\nModel Summary:")
model_rnn.summary()
```

```
--- Q4: Build & Compile an RNN Model (Simplified) ---
RNN input shape set to: (30, 1)

Building the RNN model...
Model built successfully.

Compiling the model...
Model compiled successfully with 'adam' optimizer and 'mean_squared_error' loss.

Model Summary:
/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument
  super().__init__(**kwargs)
Model: "sequential_3"
```

Layer (type)	Output Shape	Param #
simple_rnn_3 (SimpleRNN)	(None, 50)	2,600
dense_3 (Dense)	(None, 1)	51

```
Total params: 2,651 (10.36 KB)
Trainable params: 2,651 (10.36 KB)
Non-trainable params: 0 (0.00 B)
```

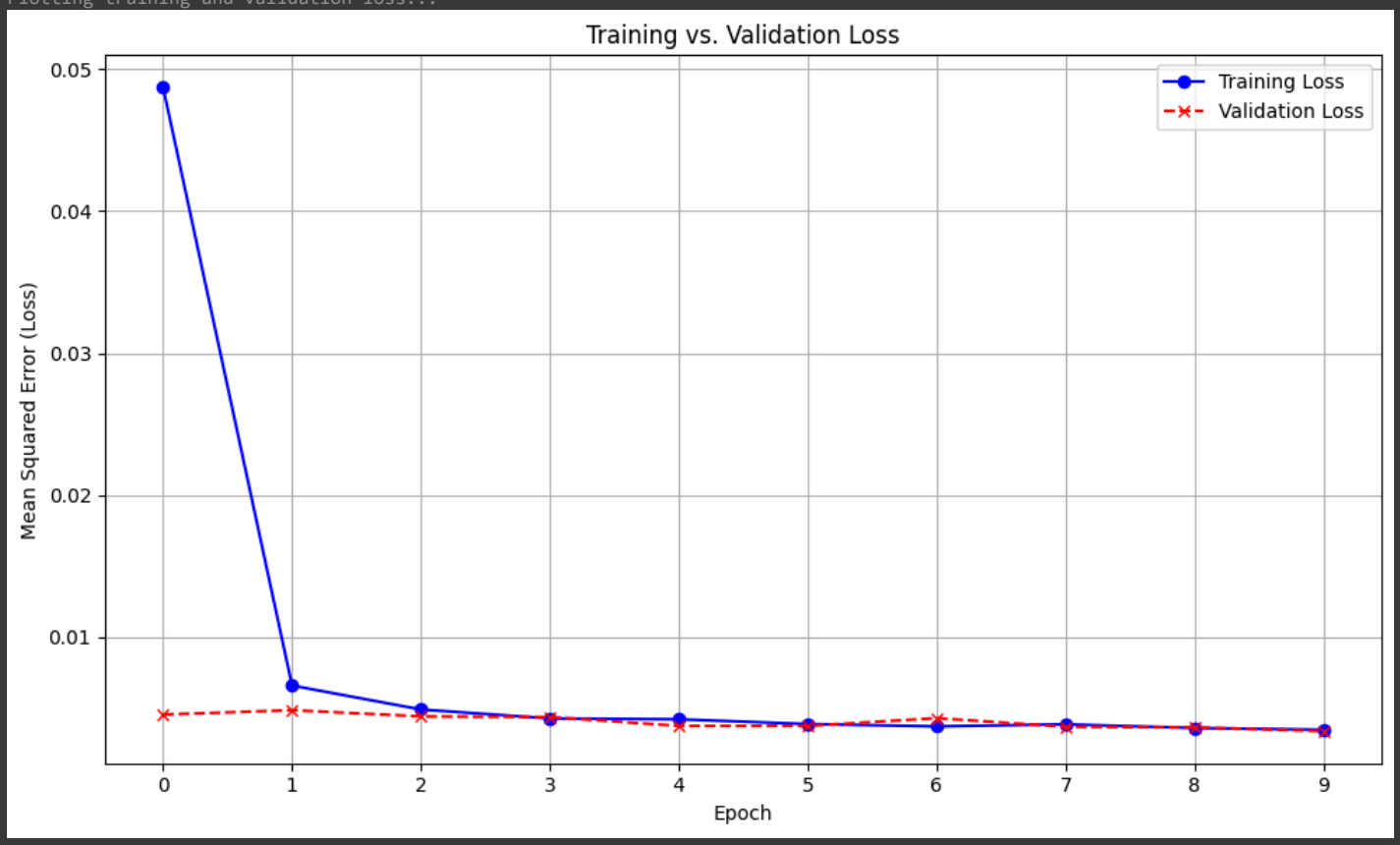
5. Train the Model & Visualize Loss

```
print("\nStarting model training...")
history = model_rnn.fit(
    x_train_sequences, y_train_sequences,
    epochs=10,
    batch_size=32,
    validation_split=0.2, # 20% of training data used for validation
    verbose=1 # Shows training progress
)
print("Model training complete.")
```

```
Starting model training...
Epoch 1/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 2s 16ms/step - loss: 0.1213 - val_loss: 0.0045
Epoch 2/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0068 - val_loss: 0.0048
Epoch 3/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0049 - val_loss: 0.0044
Epoch 4/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0044 - val_loss: 0.0044
Epoch 5/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.0047 - val_loss: 0.0037
Epoch 6/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0040 - val_loss: 0.0037
Epoch 7/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0036 - val_loss: 0.0043
Epoch 8/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0036 - val_loss: 0.0037
Epoch 9/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 0.0033 - val_loss: 0.0036
Epoch 10/10
36/36 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.0035 - val_loss: 0.0034
Model training complete.
```

```
# 3) Plotting training and validation loss
print("\nPlotting training and validation loss...")
plt.figure(figsize=(10, 6)) # Adjusted size slightly
plt.plot(history.history['loss'], label='Training Loss', color='blue', marker='o', linestyle='-')
plt.plot(history.history['val_loss'], label='Validation Loss', color='red', marker='x', linestyle='--')
plt.title('Training vs. Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Mean Squared Error (Loss)')
plt.legend()
plt.grid(True)
plt.xticks(range(0, len(history.history['loss']))) # Ensure integer ticks for epochs if less than 10-15
plt.tight_layout()
plt.show()
```

```
Plotting training and validation loss...
```



6. Evaluate Model & Plot Predictions

```
# Prepare the test data
# Extract 'meantemp' from the test set
temp_values_test = test_df['meantemp'].values.reshape(-1, 1)
```

```
print("Scaling test data...")
test_scaled = scaler.transform(temp_values_test)
print("Test data scaled.")
```

```
Scaling test data...
Test data scaled.
```

```
# Creating sequences from the scaled test data
sequence_length = 30 # Should be consistent with training
x_test_sequences, y_test_sequences_scaled = create_sequences(test_scaled, seq_length=sequence_length)

print(f"Shape of x_test_sequences: {x_test_sequences.shape}")
print(f"Shape of y_test_sequences_scaled (scaled actuals): {y_test_sequences_scaled.shape}")
```

```
Shape of x_test_sequences: (84, 30, 1)
Shape of y_test_sequences_scaled (scaled actuals): (84,)
```

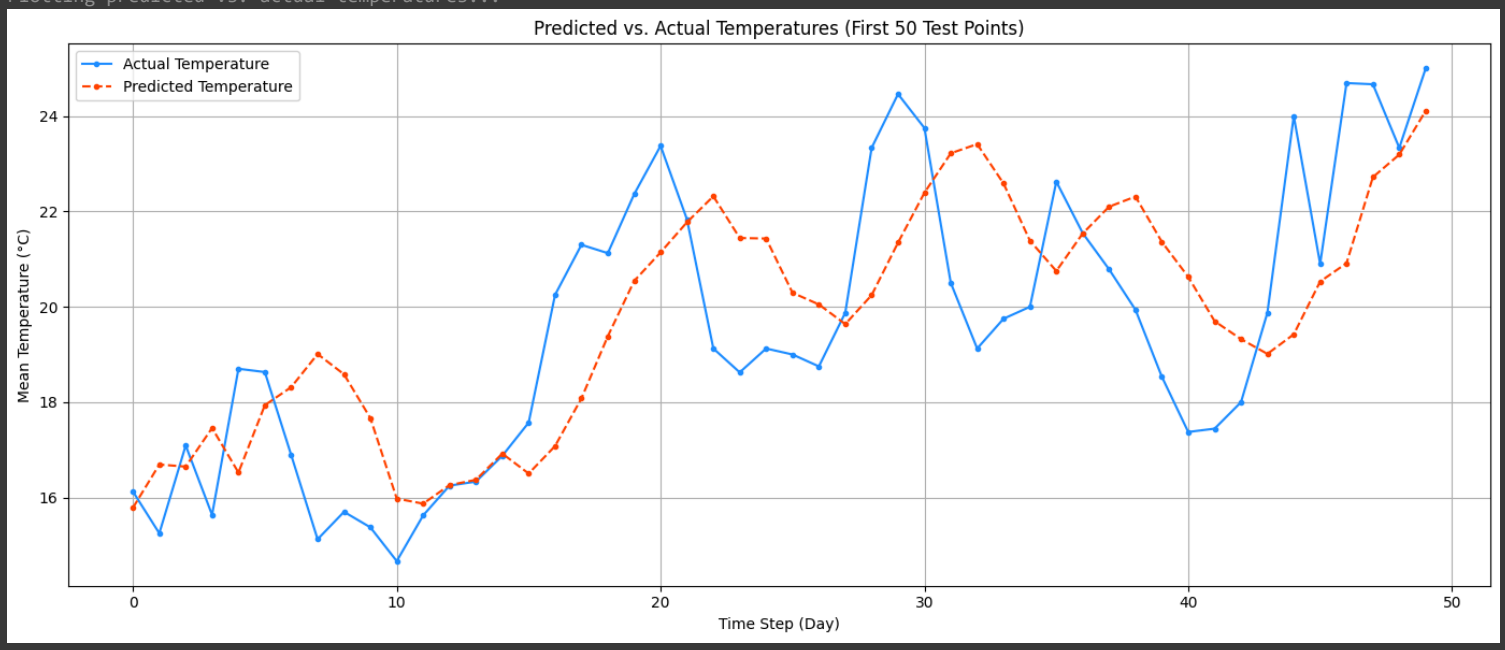
```
# 1) Predict on the test set
print("\nPredicting on the test set...")
y_pred_scaled = model_rnn.predict(x_test_sequences)
print("Prediction complete.")
print(f"Shape of y_pred_scaled (scaled predictions): {y_pred_scaled.shape}")
```

```
Predicting on the test set...
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step
Prediction complete.
Shape of y_pred_scaled (scaled predictions): (84, 1)
```

```
# Inverse transform the predicted and true values

print("\nInverse transforming predictions and actual values...")
# Predictions are already 2D (samples, 1) from model.predict()
y_pred_inverse = scaler.inverse_transform(y_pred_scaled)

# Actual test values (y_test_sequences_scaled) are 1D, need to reshape for inverse_transform
```

```
y_test_inverse = scaler.inverse_transform(y_test_sequences_scaled.reshape(-1, 1))
print("Inverse transformation complete.")
```

```
Inverse transforming predictions and actual values...
Inverse transformation complete.
```

```
# 3) Plotting predicted vs actual temperatures (first 50 points)
print("\nPlotting predicted vs. actual temperatures...")
points_to_plot = 50

plt.figure(figsize=(14, 6)) # Adjusted size slightly
plt.plot(y_test_inverse[:points_to_plot], label='Actual Temperature', color='dodgerblue', marker='.', linestyle='-')
plt.plot(y_pred_inverse[:points_to_plot], label='Predicted Temperature', color='orangered', marker='.', linestyle='--')
plt.title(f'Predicted vs. Actual Temperatures (First {points_to_plot} Test Points)')
plt.xlabel('Time Step (Day)')
plt.ylabel('Mean Temperature (°C)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
Plotting predicted vs. actual temperatures...
```